

DS5899: Special Topics in Data Science

Leveraging NLP in Asset Management

Week 7: MTP

Instructor: Che Guan, Ph.D.

Part 1:

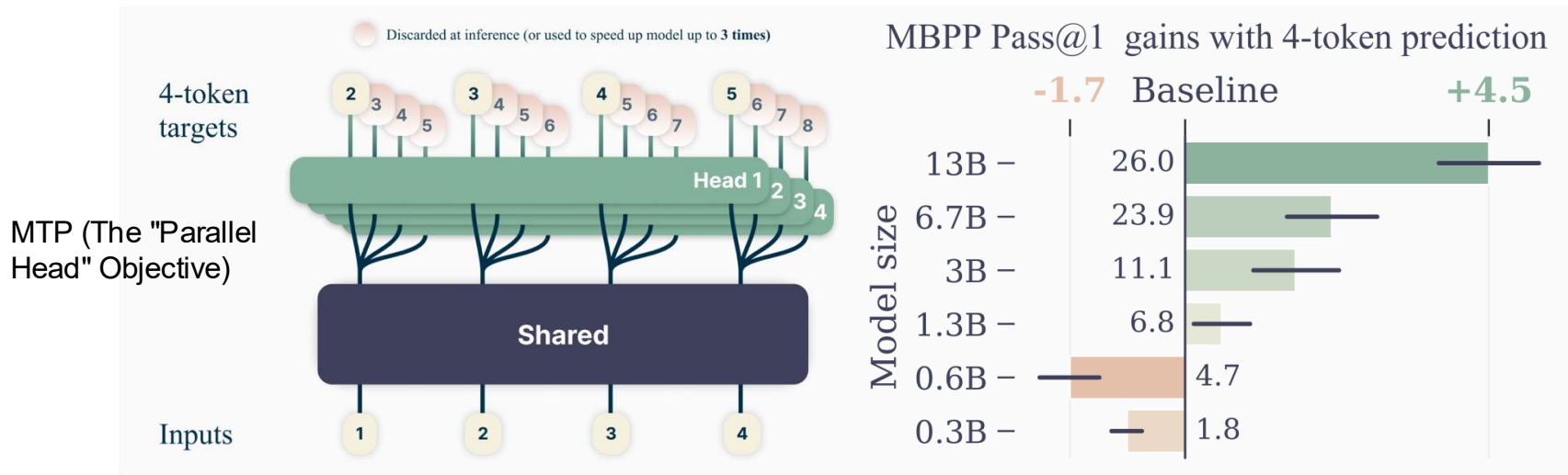
MTP (Multi-token Prediction)

From LLM NTP to LLM Planning

- Next-token prediction (NTP) is the objective function used during the pre-training of Causal (Autoregressive) Transformers. NTP primarily supports short-term coherence.
- How can we enhance model planning by encouraging models to consider future dependencies?

Multi-token Prediction (MTP)

- In the example below, during training, the model predicts 4 future tokens at once, by means of a shared trunk and 4 dedicated output heads. During inference, only the next-token output head is employed.
- MTP improves performance on the code task as model size increases.



- The predictions are statistically independent given the state of the Trunk (the shared neural network layers). The Trunk needs to produce a state vector that contains enough information to satisfy *four different futures* simultaneously

Pre-training (Teacher Forcing) versus Inference (Generation)

- During **training**, we use "Teacher Forcing." the entire gold-standard sentence is fed into the model at once. The attention mask - casual mask is used to to compute the loss for every single token in the sequence simultaneously (parallelization) without the model seeing the answers.

When calculating attention, we start with the Query (Q) and Key (K) multiplication:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T + M}{\sqrt{d_k}}\right)V$$

Where M is the mask.

- If $M_{ij} = 0$, the attention score remains unchanged.
- If $M_{ij} = -\infty$, the score becomes $-\infty$. $\emptyset$

When the **softmax** function is applied, $e^{-\infty}$ becomes 0. This effectively "zeros out" the attention weight, meaning the model assigns no importance to that future token.

Example:

For a sequence of length $N = 4$, the mask M looks like this:


$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

- During **inference**, the mask is technically still there, but since we generate tokens one by one, there are no "future" tokens yet to look at.

- 0: Allows attention (the "connection" is open).
- $-\infty$: Blocks attention (after softmax, these become 0 weight).

Speculative Decoding

- Speculative Decoding is an inference acceleration technique used to make LLMs generate text faster.
- In speculative decoding, the "magic" happens because Transformers can process multiple tokens in a **single forward pass** as long as they are treated as a sequence.
- A small model (**Approximation or Draft Model**) generates a sequence of tokens, a large model (**Target Model**) doesn't verify them one by one in a loop. Instead, it verifies the entire draft "block" simultaneously using a **Causal Attention Mask**.
 1. Draft Model generates K candidate tokens (e.g., K=5).
 2. These K tokens, plus the original prefix, are fed into the Target Model as one single sequence.
 3. The Large Model performs one forward pass

The Acceptance Logic

- The Target model (a large LLM) looks at the draft sequence $[T_1, T_2, \dots, T_K]$ from the Approximation model (a small LLM) and checks its own probability for each:
 - **Step 1:** Calculate the probability $P(T_i | \text{prefix})$ for each drafted token.
 - **Step 2:** Use a rejection sampling scheme (e.g., Greedy approach).
 - Greedy Match: If the Large Model's most likely token at position i is exactly T_i , the token is Accepted.
 - Mismatch: If at position $i = 3$ the Target Model disagrees, it Rejects T_3 and all subsequent tokens T_4, T_5).
 - **Step 3:** Target Model provides the "correct" token (e.g., correct T_3) for the first point of failure.
- What about the tokens *after* the mistake?
 - Current Iteration Ends: We accept T_1, T_2 + Target Model's T_3 .
 - Next Iteration Begins: The Draft model starts fresh, drafting a new sequence starting from the new T_3 .

Why is it Important to Use a Small LLM for Inference?

- LLM inference is **Memory-Bound**, not Compute-Bound
- To generate a single token, the GPU must "read" every single weight of the model.
 - Small Model (7B): The GPU has to move ~14GB of data (in 16-bit) from VRAM to the cores to predict one word.
 - Large Model (70B): The GPU has to move ~140GB of data to predict same word.
- Because the **Memory Bandwidth** (the rate at which data moves) is a fixed physical property of the card (e.g., 2 TB/s for an H100), it simply takes **10 times longer** to fetch the weights for the 70B model.
- The GPU's thousands of cores spend most of their time sitting idle, "starving" for data to arrive.

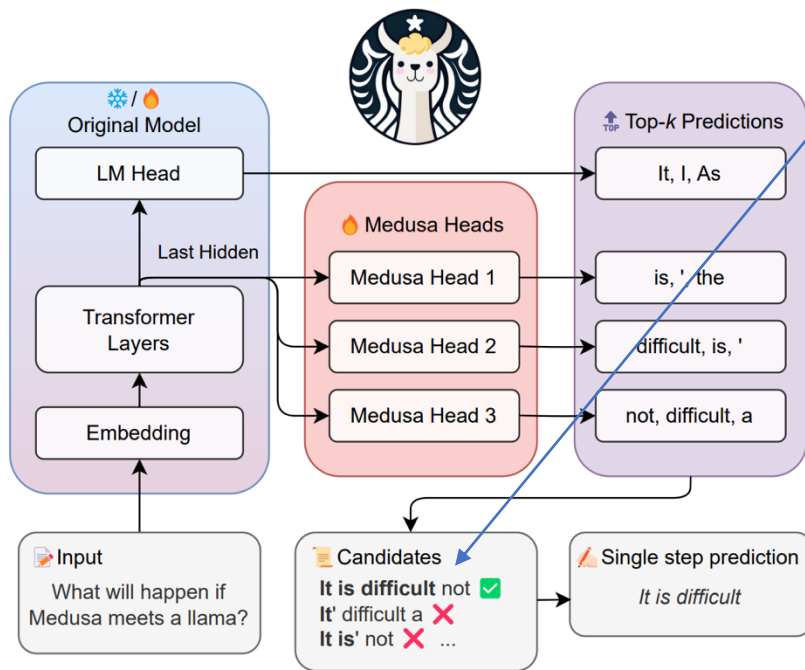
Example

```
[START] japan ' s benchmark bond n
[START] japan ' s benchmark nikkei 22 5
[START] japan ' s benchmark nikkei 225 index rose 22 6
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 0 1
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 9859
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . 79 in
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . 79 in tokyo late
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . 79 in late morning trading . [END]
```

Figure 1. Our technique illustrated in the case of unconditional language modeling. Each line represents one iteration of the algorithm. The **green** tokens are the suggestions made by the approximation model (here, a GPT-like Transformer decoder with 6M parameters trained on lm1b with 8k tokens) that the target model (here, a GPT-like Transformer decoder with 97M parameters in the same setting) accepted, while the **red** and **blue** tokens are the rejected suggestions and their corrections, respectively. For example, in the first line the target model was run only once, and 5 tokens were generated.

Medusa

- While traditional speculative decoding requires to manage two separate models (a small "Draft" model and a large "Target" model), Medusa integrates the drafting process directly into the large model itself.



Candidate Generation: The Medusa heads propose multiple candidates based on their top predictions (highest probabilities). Specifically, candidates are formed by taking the top- s_k predictions from each head.

Verification & Selection: These candidates are then verified by the original model. The system selects the longest sequence of candidates that is accepted by the backbone model.

More Details in Medusa, *Continued*

- Medusa introduces multiple heads on top of the last hidden states of the LLM, enabling the prediction of several subsequent tokens in parallel
- During inference, each head generates multiple top predictions for its designated position. These predictions are assembled into candidates, which are processed in parallel using a tree-based attention mechanism
- The final step is to verify the candidates and accept a continuation. Besides the standard rejection sampling scheme, a typical acceptance scheme can also be used here to select reasonable continuations, and the longest accepted candidate prefix is used for the next decoding phase.

Eagle vs Medusa

- **Traditional Speculative Decoding:** Uses a small model to guess tokens.
- **Medusa:** Uses simple linear heads to guess multiple future tokens.
- **EAGLE:** Uses a lightweight autoregressive head to predict the **next feature vector**. It then passes that predicted feature through the original model's "Language Model Head" to get the token.

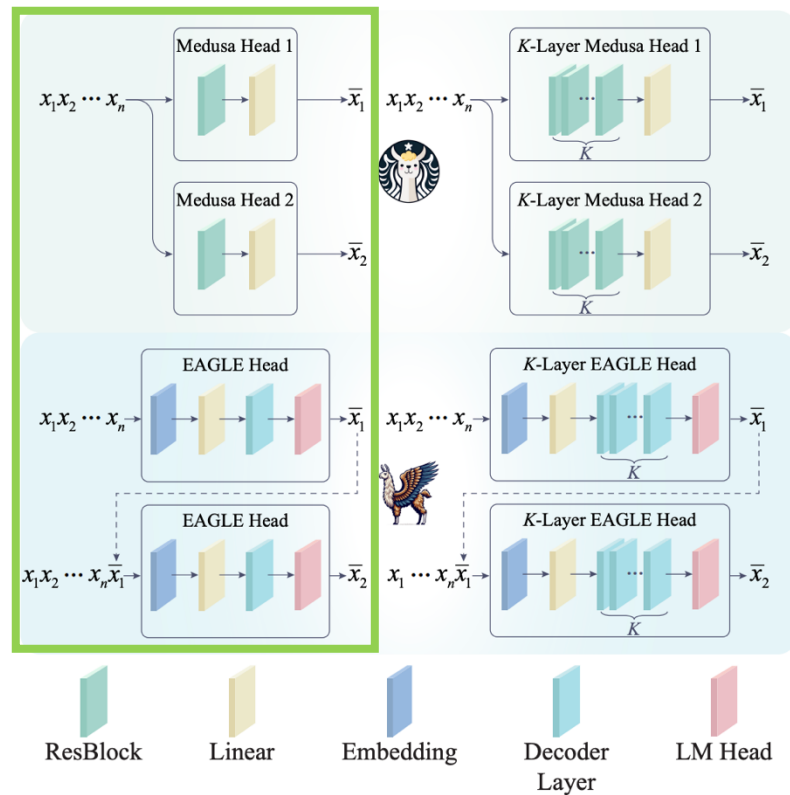


Figure 2: Comparison of single-layer and multi-layer draft head structures. For each Medusa Head, KOALA expands the single ResBlock to K layers. In the EAGLE Head, KOALA extends the single Decoder Layer to K layers. For simplicity, each draft head predicts only the next two tokens, $\bar{x}_1$ and $\bar{x}_2$, based on the input sequence $x_1, x_2, \dots, x_n$.

EAGLE

Role of the Auto-regression Head

This is the specific module responsible for the "causal" prediction step—it moves the sequence forward in time.

- **Input:** It takes the *current* feature vector (orange block, e.g., f_{can}) and the *next* token embedding (green block, e.g., e_I).
- **Action:** It processes them to predict the *next* feature vector (e.g., f_I).
- **Output:** This predicted feature is then passed to the LM Head to guess the actual word.

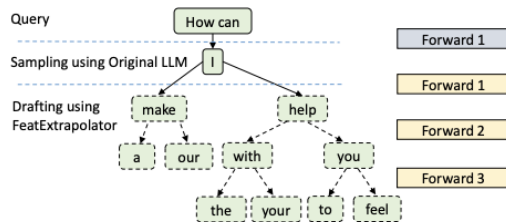
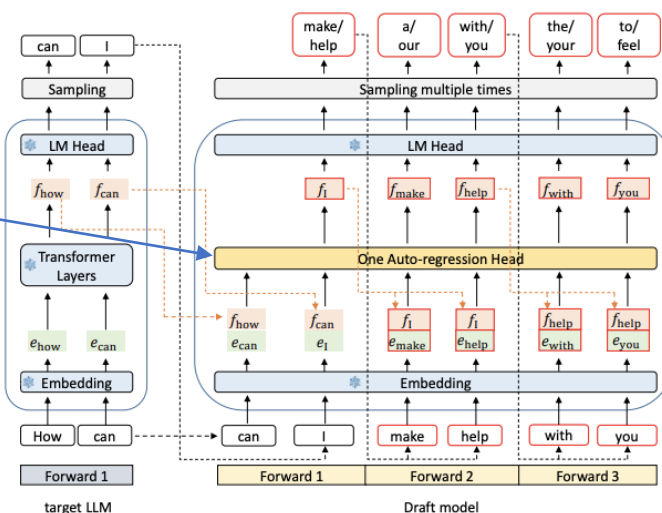


Figure 6: Pipeline of EAGLE. The upper section illustrates the computational process, while the lower section displays the corresponding generation results for each step. In the upper section, **green** blocks represent token embeddings, **orange** blocks represent features, **red** boxes indicate the predictions of the draft model, and **blue** modules with snowflake icons represent the use of target LLM parameters, which are not subject to training.

1. Phase 1: The Setup (Target LLM)

Everything starts with the original **Target LLM**.

- **Input:** The user prompts "How can".
- **Action:** The Target LLM processes this prompt. It computes the "ground truth" feature for the last word (f_{can}) and samples the very first new token: "I".
- **Handoff:** It passes the token "I" and the feature f_{can} to the Draft Model.

Initial Sampling

2. Phase 2: The Drafting (Draft Model)

Now the lightweight **Draft Model** takes over to build the tree of future possibilities.

- **Step 1: Predicting "help" / "make"**
 - **Input:** The model takes the feature from the Target (f_{can}) and the embedding of the new token (e_I).
 - **Prediction:** The Auto-regression Head predicts the next feature (f_I).
 - **Branching:** The LM Head uses f_I to guess the next likely words. It creates two branches: "help" (high probability) and "make" (lower probability).
- **Step 2: Predicting "you" / "with" (and others)**
 - The model now expands these branches further using its own previous predictions.
 - **For the "help" branch:** It takes its predicted feature f_I and the embedding e_{help} to predict f_{help} . From this, it guesses 6 options: "with", "you", "the", "your", "to", "feel".
 - **For the "make" branch:** It takes f_I and e_{make} to predict f_{make} . From this, it guesses 2 options: "a", "our".

Tree Generation

3. Phase 3: Verification (Target LLM)

Finally, the **Target LLM** returns to check the Draft Model's work.

- **Input:** It takes the entire "Draft Tree" (containing "I help you", "I help with", "I make a", etc.) as a single input.
- **Action:** It runs one forward pass to calculate the true probabilities for every node in the tree simultaneously.
- **Selection:**
 - It confirms "I" is correct.
 - It looks at the "help/make" split and confirms "help" (rejecting "make").
 - It looks at the "you/with/the..." split and confirms "you" (rejecting the others).
- **Result:** The valid sequence "I help you" is appended to the output.

DeepSeek V3- MTP

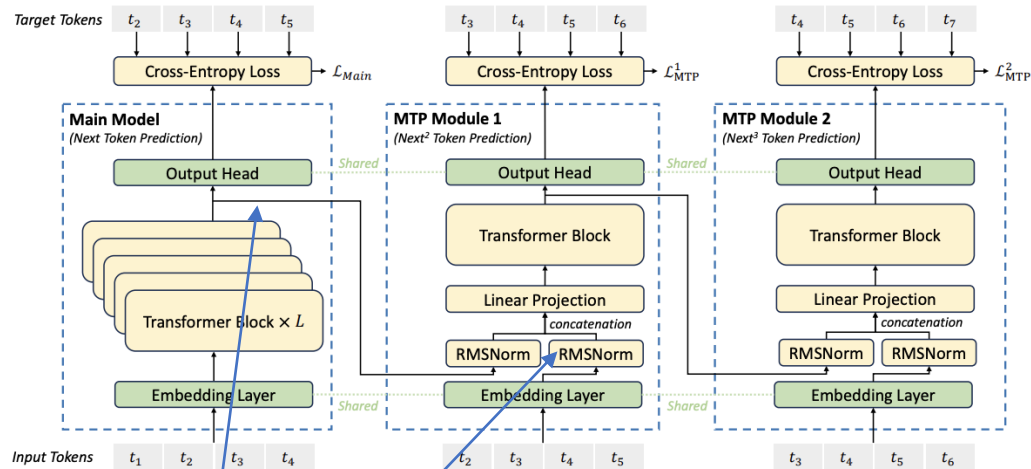


Figure 3 | Illustration of our Multi-Token Prediction (MTP) implementation. We keep the complete causal chain for the prediction of each token at each depth.

- **Output Head** takes the feature representation generated by the Transformer blocks to predict the next token.
- **MTP Module 1** concatenates the feature representation (output) from the Main Model (which processed the input token t_1) and the embedding of the *next* token t_2 to predict the *subsequent* token t_3 , repeat the process.
- The main goal of EAGLE is speculative decoding, which focuses on accelerating inference speed. The primary goal of MTP in V3 is to improve training, the MTP modules can be discarded during inference.

1. The Main Model (Left) The left side of the diagram shows the standard "Main Model." It functions like a typical Large Language Model (LLM): it takes an input (like token t_1) and processes it through Transformer blocks to predict the immediate next token (t_2). This is the standard "Next Token Prediction" task. [Prediction](#)

2. MTP Modules (Middle and Right) To the right of the main model, you see "MTP Module 1" and "MTP Module 2." These are additional, smaller blocks appended to the model.

- **MTP Module 1** is responsible for predicting the *second* future token (e.g., predicting t_3 based on the context of t_1 and t_2).
- **MTP Module 2** predicts the *third* future token.
- This approach maintains a "causal chain," meaning the model uses the information from the first prediction step to help make the second prediction, rather than guessing them all independently.

[Causal Chain](#)

3. Shared Components The figure highlights that certain parts of the architecture are reused to maintain efficiency. Specifically, the **Embedding Layer** (which converts text into numbers) and the **Output Head** (which converts numbers back into predicted probabilities) are shared across the main model and the MTP modules. [Shared Components](#)

EAGLE-3

- EAGLE-3 (Direct Token Prediction): found that "feature prediction" from EAGLE actually acted as an unnecessary constraint.

1. l_{how} (Low-Level Feature)

- Location:** Extracted from a shallow (early) layer of the Target Model.
 - Content:** Typically captures basic lexical information (e.g., grammar, syntax, surface-level word meaning) of the word "How".
- Feature definition

2. m_{how} (Middle-Level Feature)

- Location:** Extracted from an intermediate (middle) layer of the Target Model.
 - Content:** Captures a mix of syntax and semantics, bridging the gap between raw words and deep meaning.
- Feature extraction

3. h_{how} (High-Level Feature)

- Location:** Extracted from the top (final) layer of the Target Model (often the layer just before the LM Head).
 - Content:** Captures the deepest semantic understanding and context of the word "How" within the sentence. In previous versions (like EAGLE-1), this was the *only* feature used.
- Top-layer usage

Why gather all three? (Multi-Layer Feature Fusion)

The key innovation in EAGLE-3 shown in Figure 5 is that it doesn't just trust the top layer (h). It concatenates all three (l, m, h) and fuses them through a Fully Connected (FC) layer to create the master vector g_{how} .

$$\text{Target Features}(l, m, h) \xrightarrow{\text{Concat} + \text{FC}} g_{how}$$

This ensures the Draft Model gets a complete picture of the token—from its basic grammar (l) to its complex meaning (h)—which helps it make better predictions.

Fusion mechanism

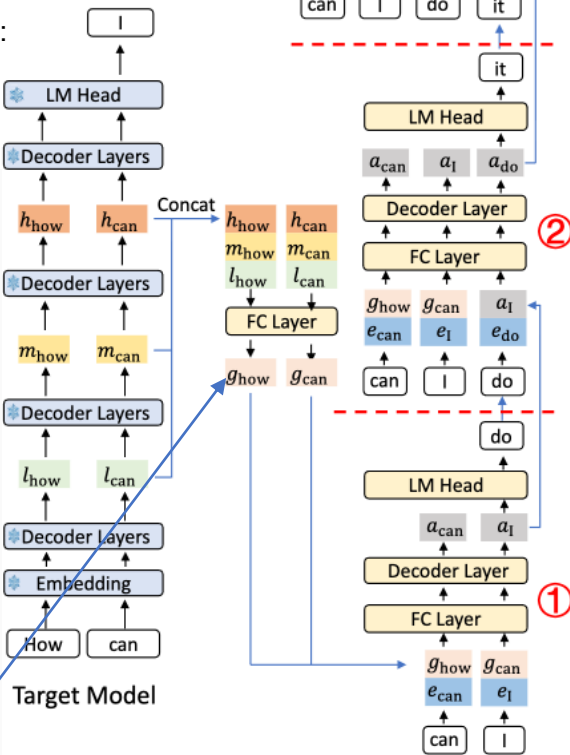


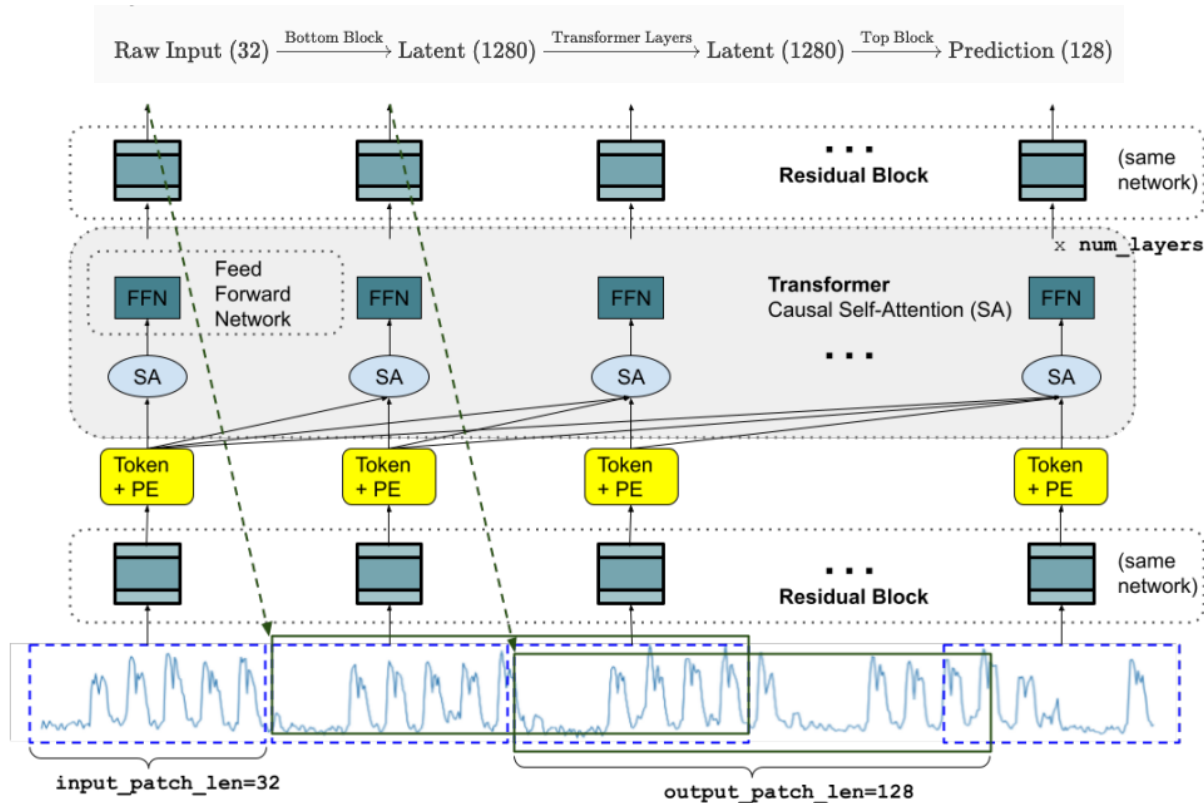
Figure 5: Diagram of the EAGLE-3 inference pipeline, illustrating the three steps of the draft model. l, m , and h represent the low, middle, and high-level features of the target model, respectively. e denotes the embedding.

Part 2:

Foundation Model For Time-Series Forecasting

TimesFM (Time Series Foundation Model)

- TimesFM is a decoder-only transformer model designed for univariate time-series forecasting that excels at 0-shot inference.



TimesFM - Patching

- How does Patch strategy work?

What is the benefit of patch masking (r)?

- Here is an example for the TimesFM 200M model setup ($p=32$, $r=3$, max context=512) and output patch size (h) = 128.

TimesFM 200M Model Configuration

Parameter	Value	Description
Context Window Size	512	Max history length seen during training
Number of Patches	16	Effective sequence length ($512/32$)
Input Patch Size (p)	32	Points per input token
Masking Parameter (r)	0-31	Random offset for first patch masking
Model Dimension (d_{model})	1280	Hidden state size in Transformer layers
Attention Heads	16	Number of parallel attention heads
Head Dimension	80	Size per head ($1280/16$)
Layers	20	Number of Transformer blocks
Output Patch Size (h)	128	Points predicted per step

We mask the first 3 points of the first patch (points 1-3 are hidden).

- After Patch 1 (Input length = 29):
 - Input (Visible Past): Points 4-32 (29 points)
 - Target (Next Output Patch): Points 33-160 (128 points)
- After Patch 2 (Input length = 61):
 - Input (Visible Past): Points 4-64 (61 points total: 29 from patch 1 + 32 from patch 2)
 - Target (Next Output Patch): Points 65-192 (128 points)
- After Patch 3 (Input length = 93):
 - Input (Visible Past): Points 4-96 (93 points total)
 - Target (Next Output Patch): Points 97-224 (128 points)

This continues for all 16 patches, allowing the model to train on context lengths like 29, 61, 93... rather than just multiples of 32.

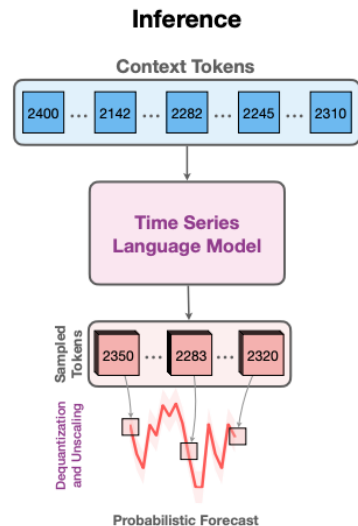
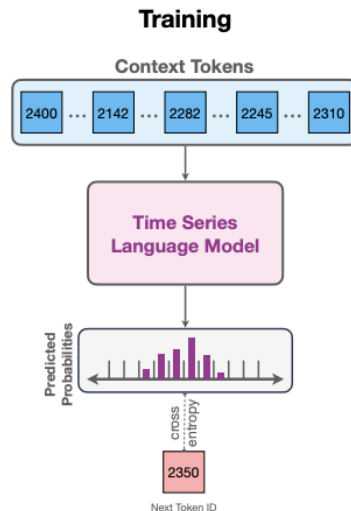
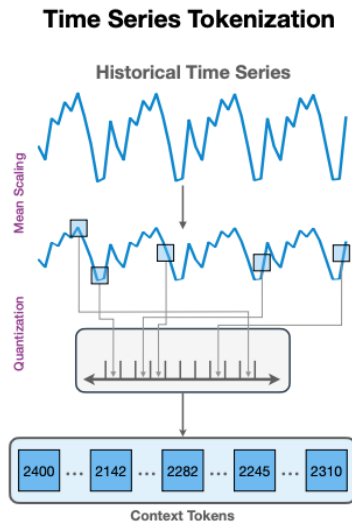
This is one of the key efficiency benefits of the "patching" approach.

- Without Patching:** If the model attended to every time point individually, the matrix would be 512×512 (262,144 entries).
- With Patching:** It only attends to the 16 patch tokens, resulting in 16×16 matrix (256 entries).

The 16 patches are exactly what the Transformer "sees" as the **actual tokens** during the attention process!

Chronos

- Amazon Chronos uses a T5-based transformer architecture for time-series forecasting. It treats time-series data as language by scaling and discretizing numerical values into tokens, enabling probabilistic forecasts with high zero-shot accuracy.



1. Left Panel: Input Processing (Forward Flow)

- Raw Data → Scaling:** The first arrow takes the raw time series values (e.g., 2400, 2282) and passes them through **Mean Scaling**. This normalizes the data so the model can handle different magnitudes (e.g. sales vs. temperature) uniformly.
- Scaling → Quantization:** The next arrow moves the scaled values into **Quantization**. This maps the continuous numbers into discrete "bins."
- Quantization → Tokens:** The result is a sequence of **Tokens** (integers like 342, 51, 99), which are the actual inputs the Language Model can understand.

2. Center Panel: Training

- Tokens → Language Model:** The arrows feed the sequence of tokens into the **Transformer** architecture.
- Language Model → Cross-Entropy Loss:** The model outputs a prediction for the *next* token. The arrow to the loss function compares this prediction against the *actual* next token to update the model weights.

3. Right Panel: Inference (Reverse Flow)

- Language Model → Distribution:** The arrow shows the model outputting a probability distribution over all possible tokens (bins) for the next time step.
- Distribution → Sampling:** An arrow selects (samples) one specific token from that distribution (e.g., token ID 55).
- Sampling → Dequantization:** This arrow starts the reverse process. It turns the discrete token ID back into a generic scaled value (e.g., 0.8).
- Dequantization → Unscaling:** The final arrow applies the **Inverse Mean Scaling** (multiplying by the original scale factor) to convert the value back to the original unit (e.g., converting 0.8 back to 2150).

Chronos - Time Series Tokenization

The Problem: Predict Tomorrow's Sales

We have sales for 3 days and want to predict **Day 4**.

- **Input Data:** [100, 200, 50]
- **Target (Day 4):** 150

Step 1: Scaling (Find the Average)

We calculate the average of the **context** (the input data).

- $\text{Sum} = 100 + 200 + 50 = 350$
- $\text{Average} = 350/3 \approx 117$

Step 2: Scale Everything (Including the Target)

Divide all numbers by 117.

- Day 1: $100/117 \approx 0.85$
- Day 2: $200/117 \approx 1.71$
- Day 3: $50/117 \approx 0.43$
- **Target (Day 4):** $150/117 \approx 1.28$

Step 3: Create Buckets (The Vocabulary)

We define our bins (tokens).

- **Token 1:** 0.0 — 0.5
- **Token 2:** 0.5 — 1.0
- **Token 3:** 1.0 — 1.5
- **Token 4:** 1.5 — 2.0

Step 4: Tokenize (Drop into Buckets)

- **Input Sequence:**
 - 0.85 → Token 2
 - 1.71 → Token 4
 - 0.43 → Token 1
 - **Final Input:** [2, 4, 1]
- **Target Token:**
 - 1.28 → Token 3

Step 5: Training

The model sees [2, 4, 1] and outputs probabilities for the next token.

- **Model Prediction:** Let's say it predicts **Token 3** with 80% probability.
- **Loss Calculation:** Since the **Target Token** is actually **3**, the model is correct! The loss is low.

Time Series Tokenization - Inference

- Let's extend the previous lemonade stand example to include the Inference (Forecasting) steps: Dequantize and Unscale

Recap:

- Original History: [100, 200, 50]
- Scale Factor (Mean): 117
- Bins (Tokens):
 - Token 1: 0.0 — 0.5
 - Token 2: 0.5 — 1.0
 - Token 3: 1.0 — 1.5
 - Token 4: 1.5 — 2.0

Step 6: Sampling (Pick a Token)

The model outputs a probability distribution for the next day.

- Let's say it gives **80% probability to Token 3** and 20% to Token 2.
- We sample from this distribution and pick **Token 3**.

Step 8: Unscale (Scaled Value → Real Dollars)

Finally, we multiply by our original **Scale Factor (117)** to get the actual sales forecast.

- Calculation: 1.25×117
- Result: 146.25

Step 7: Dequantize (Token → Scaled Value)

We need to turn **Token 3** back into a number. Since Token 3 represents the range 1.0 — 1.5, we pick the **center value** of that bin.

- Center of Token 3: $(1.0 + 1.5)/2 = 1.25$
- Predicted Scaled Value: 1.25

Final Forecast

The model predicts **\$146.25** in sales for Day 4.

Note: This is slightly different from the original target of 150 because we used the *center* of the bin, losing a little precision. This is a trade-off of quantization

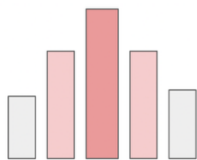
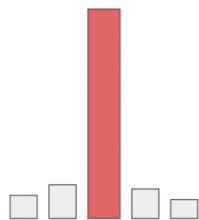
Part 3:

Decoding Strategies

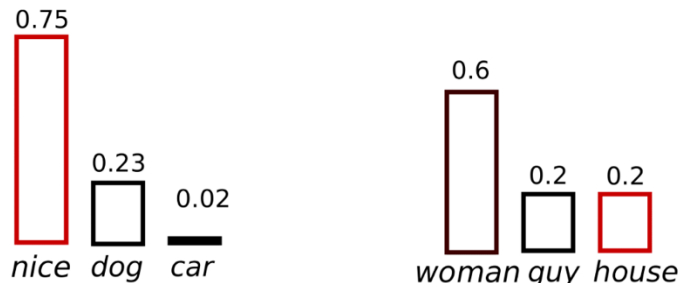
Temperature

- An effective technique involves adjusting the temperature of the softmax to make the probability distribution sharper. By reducing the temperature, the likelihood of high probability words increases while the likelihood of low probability words decreases. This trick helps improve the generation process.

Low temperature vs High temperature



The difference between the low-temperature case (left) and the high-temperature case for the categorical distribution is illustrated in the picture above, where the heights of the bars correspond to probabilities.



The

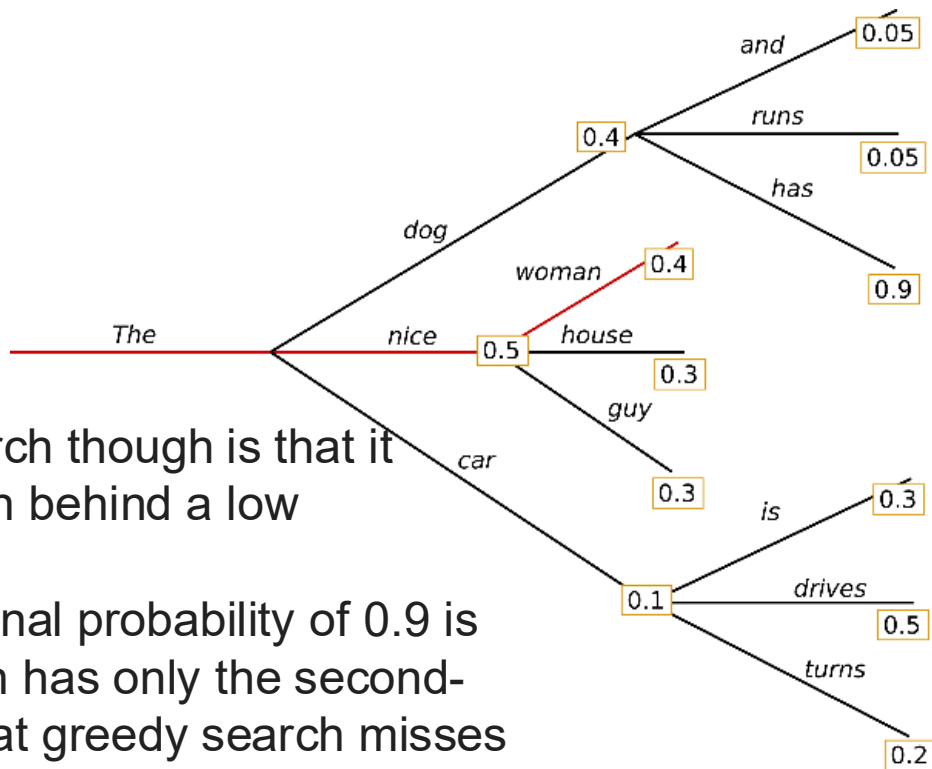
nice

house

- When the temperature is set to a very low value, such as in the limit of temperature $\rightarrow 0$, the conditional next word distribution at step $t = 1$ becomes significantly sharper, leaving almost no chance for word "car".
- While temperature scaling can reduce randomness in the distribution, it is important to note that when temperature approaches 0, the sampling process becomes equivalent to **Greedy Decoding**. This can reintroduce the issues encountered previously, leading to potential limitations in the generated output.

Greedy Decoding

- Starting from the word "The", the algorithm greedily chooses the next word of highest probability "nice" and so on, so that the final generated word sequence is {"The", "nice", "woman"} having an overall probability of $0.5 \times 0.4 = 0.2$.
- The major **drawback** of greedy search though is that it misses high probability words hidden behind a low probability word.
- The word "has" with its high conditional probability of 0.9 is hidden behind the word "dog", which has only the second-highest conditional probability, so that greedy search misses the word sequence "The", "dog", "has".



Greedy Decoding, *Continued*

- Let's see how greedy search can be used in `GP2` as follows. we will generate word sequences using GPT2 on the context `{"I", "enjoy", "walking", "with", "my", "cute", "dog"}`.

```
1 import tensorflow as tf
2 from transformers import TFGPT2LMHeadModel, GPT2Tokenizer
3
4 tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
5
6 # add the EOS token as PAD token to avoid warnings
7 model = TFGPT2LMHeadModel.from_pretrained("gpt2", pad_token_id=tokenizer.eos_token_id)
8
9 # encode context the generation is conditioned on
10 input_ids = tokenizer.encode('I enjoy walking with my cute dog', return_tensors='tf')
11
12 # generate text until the output length (which includes the context length) reaches 50
13 greedy_output = model.generate(input_ids, max_length=50)
14
15 print("Output:\n" + 100 * '-')
16 print(tokenizer.decode(greedy_output[0], skip_special_tokens=True))
```

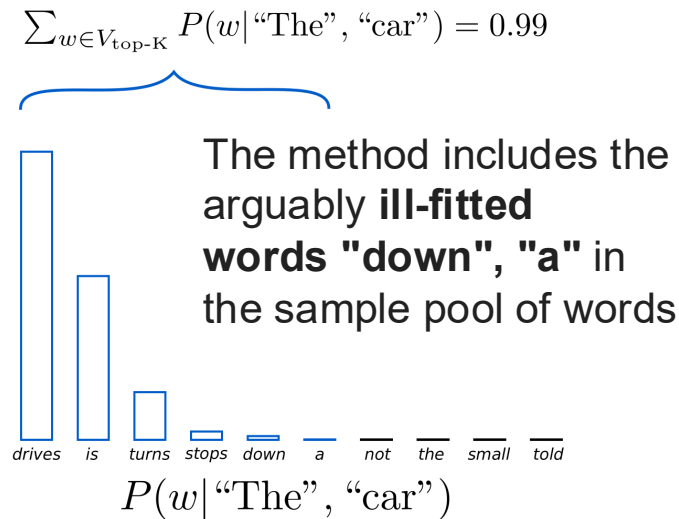
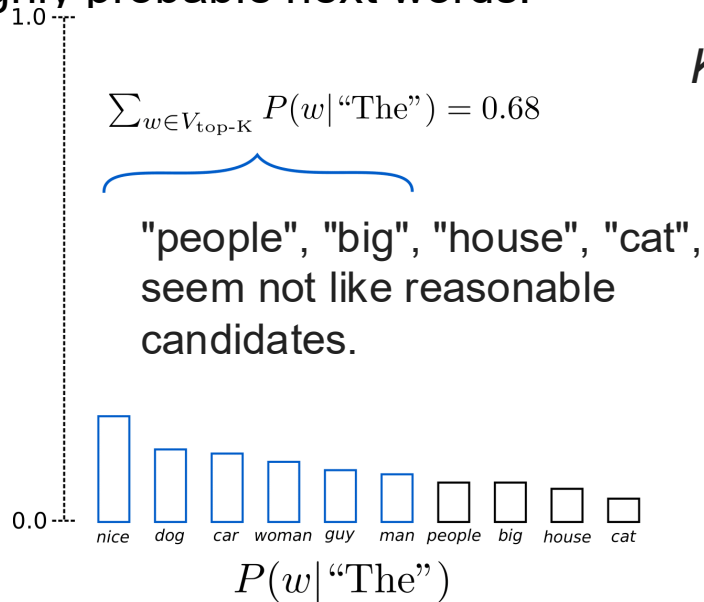
Output:

```
I enjoy walking with my cute dog, but I'm not sure if I'll ever be able to walk with my dog. I'm not sure if I'll ever be able to walk with
I'm not sure if I'll
```

- The generated words following the context are reasonable, but the model quickly starts **repeating** itself. One major reason is attributed to the fact that there exist too many words predicting the same word as the subsequent word with high probability. Consequently, it is easy to go back to that word and form repetitions, and we dub it as the high inflow problem (Fu ,et, al, 2021)

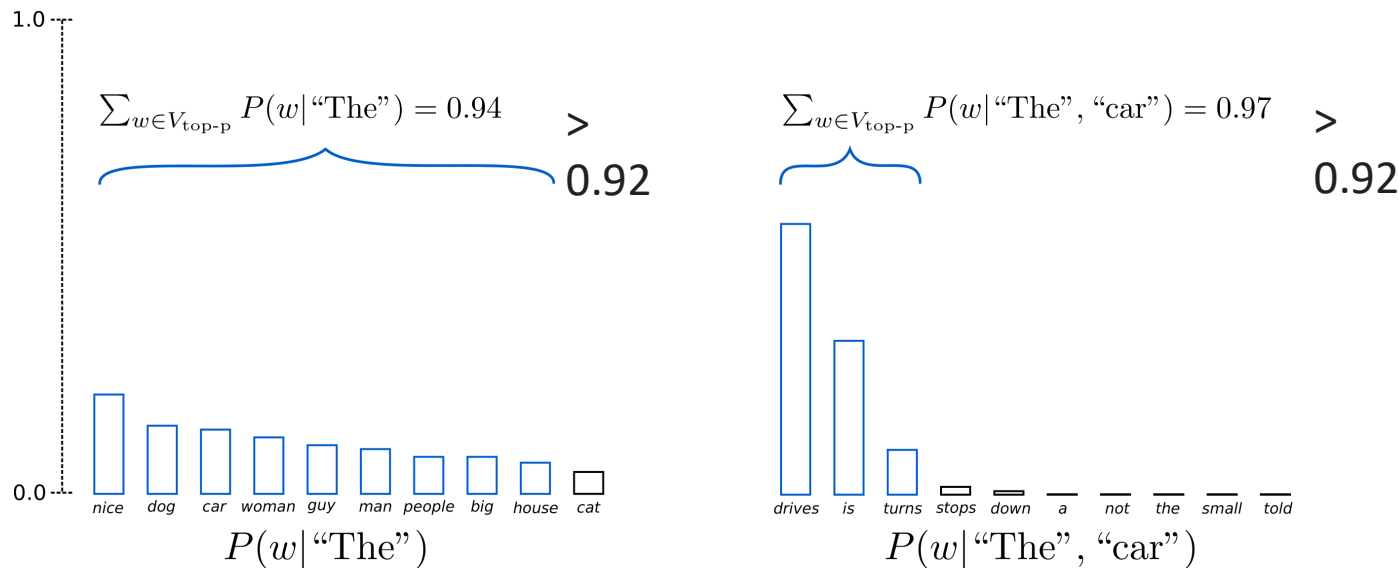
Top-K Sampling

- GPT-2 adopted a sampling scheme called Top-K sampling, where the K most likely next words are selected, and the probability mass is redistributed solely among those K options.
- This technique helps to focus the sampling process on a smaller set of highly probable next words.



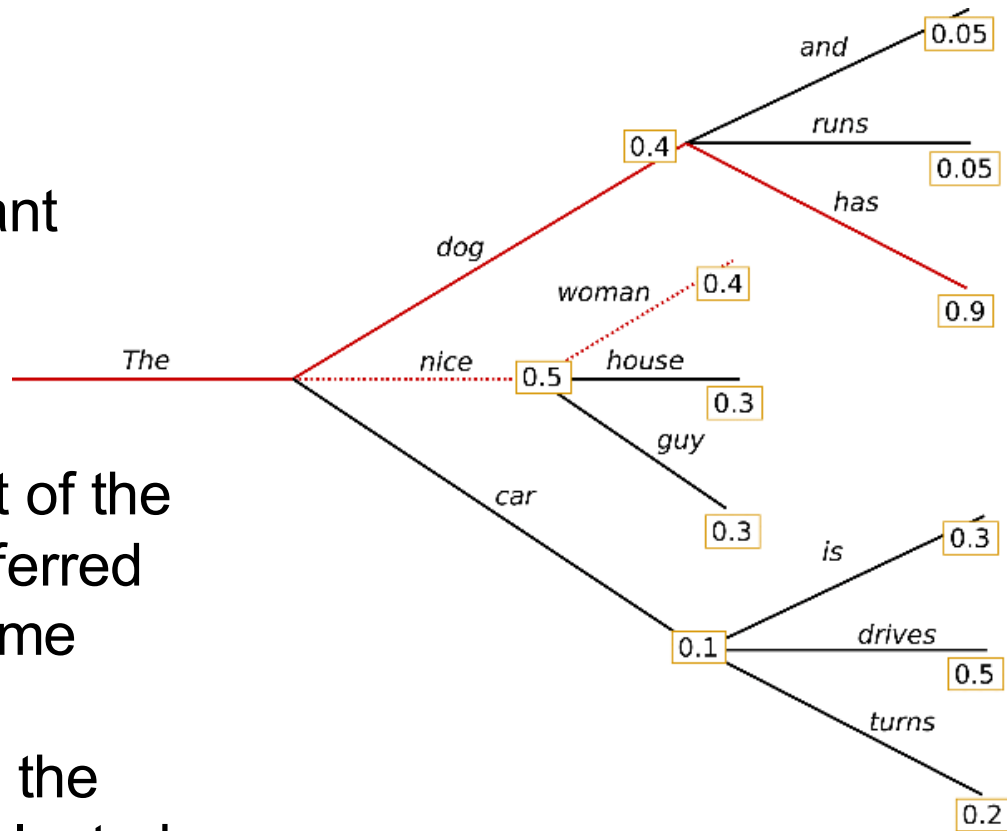
Top-p (Nucleus) Sampling

- With a chosen probability threshold of $p=0.92$, Top-p sampling selects the minimum number of words needed to accumulate a cumulative probability mass of at least 92%.
- In the example, it involved selecting the 9 most likely words, while in the 2nd example, only the top 3 words were sufficient to exceed 92% threshold.



Beam Search

- To mitigate the possibility of overlooking potentially important word sequences with high probabilities, beam search is employed.
- This technique maintains a set of the most probable hypotheses, referred to as "num_beams," at each time step.
- Ultimately, the hypothesis with the highest overall probability is selected as the final output.



e.g., num_beams = 2

Beam Search versus Pure Sampling

- Beam Search, which focuses on maximizing probability, results in degenerate repetition. Instead of progressing the narrative, the model gets stuck in a loop, endlessly repeating the name of a university.
- Conversely, Pure Sampling avoids this repetition but produces text that is incoherent and nonsensical. The output drifts into "gibberish" that is grammatically loose and logically disconnected from the initial context, inventing terms like "Bolivian Cavalleros."

Context: In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

Beam Search, $b=32$:

"The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the Universidad Nacional Autónoma de México (UNAM) and the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de ..."

Pure Sampling:

They were cattle called **Bolivian Cavalleros**; they live in a remote desert **uninterrupted by town**, and they speak **huge, beautiful, paradisiacal Bolivian linguistic thing**. They say, 'Lunch, marge.' They don't tell what the lunch is," director Professor Chuperas Omwell told Sky News. "They've only been talking to scientists, like we're being interviewed by TV reporters. We don't even stick around to be interviewed by TV reporters. Maybe that's how they figured out that they're cosplaying as the Bolivian Cavalleros."

Beam Search, *Cont.*

```
1 # activate beam search and early_stopping
2 beam_output = model.generate(
3     input_ids,
4     max_length=50,
5     num_beams=5,
6     early_stopping=True
7 )
8
9 print("Output:\n" + 100 * '-')
10 print(tokenizer.decode(beam_output[0], skip_special_tokens=True))
```

Output:

```
I enjoy walking with my cute dog, but I'm not sure if I'll ever be able to walk with him again.
I'm not sure if I'll ever be able to walk with him again. I'm not sure if I'll
```

```
1 # set no_repeat_ngram_size to 2
2 beam_output = model.generate(
3     input_ids,
4     max_length=50,
5     num_beams=5,
6     no_repeat_ngram_size=2,
7     early_stopping=True
8 )
9
10 print("Output:\n" + 100 * '-')
11 print(tokenizer.decode(beam_output[0], skip_special_tokens=True))
```

Output:

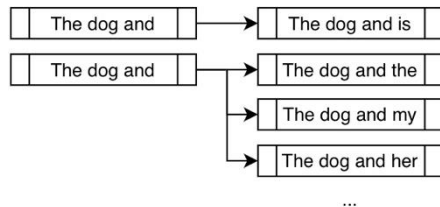
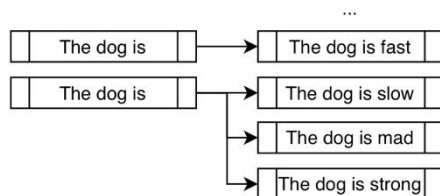
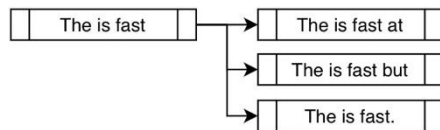
```
I enjoy walking with my cute dog, but I'm not sure if I'll ever be able to walk with him again.
I've been thinking about this for a while now, and I think it's time for me to take a break
```

- To ensure completion of generation when all beam hypotheses reach the EOS token, we utilize a value of `num_beams` greater than 1 and set `early_stopping` to `True`
- The output still includes repetitions of the same word sequences.
- An effective solution is to apply penalties to n-grams (word sequences of n words) to prevent repetition. By setting `no_repeat_ngram_size` to 2, we can ensure that no 2-gram appears more than once. Let's test this approach.

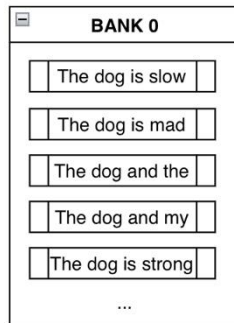
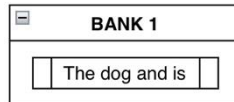
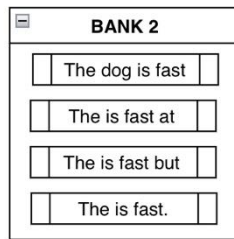
Constrained Beam Search

- Constrained beam search provides a versatile approach to incorporate external knowledge and requirements into text generation.
- In this specific example, our objective is to ensure the phrase **"is fast"** appears in the generated output. To achieve this, we not only consider high-probability next tokens like "dog" and "nice," but we also enforce the token "is" to bring us closer to fulfilling the constraint of "is fast."
- In this context, "BANK n" is a structural component of the beam that categorizes hypotheses based on how many pre-specified constraints (keywords or phrases) they have successfully satisfied.

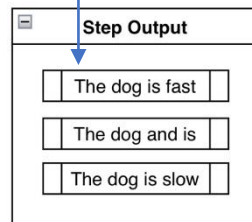
"The is fast" doesn't require any manual appending of constraint tokens since it's already fulfilled, but blindly appending the desired tokens led to nonsensical



Bank 2: The final bank, containing sequences that have satisfied **all** required constraints.



we ended up at a sensible output that contains our constraint phrase



beams like **"The dog is slow"** or **"The dog is mad"** are in Bank 0, since, although it includes the token "is", it must restart from the beginning to generate "is fast". By appending something like "slow" after "is", it has effectively reset its progress.

Bank 0: Contains hypotheses that have met **zero** constraints.

Comparison

Beam Search	Random Sampling
Maximize total probability.	Increase diversity/creativity.
Repetition and "dull" text.	Inconsistency and gibberish.

Other Sampling methods

Feature	Temperature (T)	Top-K	Top-P (Nucleus)
Core Mechanism	Rescales the entire probability distribution.	Truncates the list to a fixed number of words.	Truncates the list based on a cumulative probability threshold.
Low Setting ($T < 0.7, K < 10, P < 0.5$)	Risk: Repetition. Makes the distribution "sharp," forcing the model to pick only the #1 choice.	Risk: Generic Text. Limits the model to a tiny, "safe" vocabulary.	Risk: Rigidity. Only allows the most mathematically certain words.
High Setting ($T > 1.2, K > 100, P > 0.95$)	Risk: Inconsistency. Flattens the distribution; low-probability "gibberish" becomes likely.	Risk: Randomness. Allows the model to pick from a massive "tail" of unlikely words.	Risk: Looseness. Includes too many "unlikely" candidates when the model is uncertain.

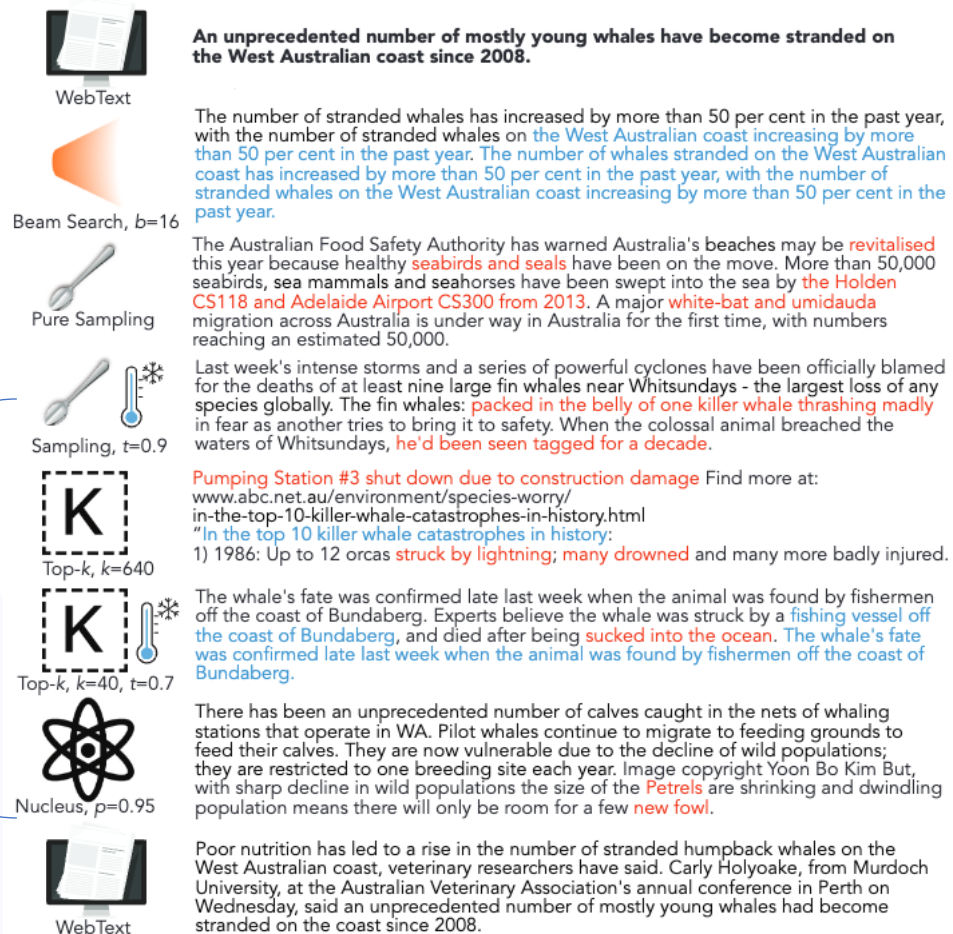


Figure 3: Example generations continuing an initial sentence. Maximization and top- k truncation methods lead to copious repetition (highlighted in blue), while sampling with and without temperature tends to lead to incoherence (highlighted in red). Nucleus Sampling largely avoids both issues.

Combine *Top-k* and *Top-p*

- In practice, both Top-p and Top-K sampling methods are effective, offering different advantages. Moreover, a combination of Top-p and Top-K can be utilized, providing the benefit of avoiding very low ranked words while allowing for dynamic selection based on the specific requirements of the task or context.

```
1 # set seed to reproduce results. Feel free to change the seed though to get different results
2 tf.random.set_seed(0)
3
4 # set top_k = 50 and set top_p = 0.95 and num_return_sequences = 3
5 sample_outputs = model.generate(
6     input_ids,
7     do_sample=True,
8     max_length=50,
9     top_k=50,
10    top_p=0.95,
11    num_return_sequences=3
12 )
13
14 print("Output:\n" + 100 * '-')
15 for i, sample_output in enumerate(sample_outputs):
16     print("{}: {}".format(i, tokenizer.decode(sample_output, skip_special_tokens=True)))
```

Output:

```
-----
0: I enjoy walking with my cute dog. It's so good to have the chance to walk with a dog. But I have this problem with
the dog and how he's always looking at us and always trying to make me see that I can do something
1: I enjoy walking with my cute dog, she loves taking trips to different places on the planet, even in the desert!
The world isn't big enough for us to travel by the bus with our beloved pup, but that's where I find my love
2: I enjoy walking with my cute dog and playing with our kids," said David J. Smith, director of the Humane Society
of the US.
"So as a result, I've got more work in my time," he said.
```

- Top-K removes the "bottom of the barrel" entirely, while Top-P adjusts the window based on how certain the model is about the next word.
- If the model is very confident, Top-P might only pick 1 or 2 tokens from the Top-K set. If it's uncertain, it will use more of the Top-K set.